

CO4 – AT1

Coding Challenge Task

REPORT:

Unbounded Knapsack Problem Using Dynamic Programming

Aim

To implement the Unbounded Knapsack problem using Dynamic Programming and determine the maximum possible profit when each item can be selected any number of times without exceeding the given capacity.

Problem Statement

In the Unbounded Knapsack problem, there are n types of items. Each item has a weight and a corresponding profit/value.

Unlike the 0/1 Knapsack problem, an item can be selected multiple times.

The objective is to maximize the total profit while ensuring that the total weight does not exceed the knapsack capacity.

Input

The input consists of:

- n = Number of item types.
- $\text{Values}[i]$ = Profit/value of item i .
- $\text{Weights}[i]$ = Weight of item i .
- W = Maximum capacity.

Sample Input

$n = 3$

Values = 10 30 20

Weights = 5 10 15

$W = 100$

Output

The program outputs:

Maximum Profit = 300

Difference Between 0/1 and Unbounded Knapsack

0/1 Knapsack	Unbounded Knapsack
Each item can be used once	Each item can be used multiple times
Item is either selected or not selected	Item can be repeatedly selected
Limited item availability	Unlimited item availability
Uses 0/1 choice	Allows repeated selection

Dynamic Programming Approach

A one-dimensional DP array is used.

Let:

$dp[w]$ = Maximum profit possible with capacity w

Initially:

$dp[0] = 0$

For every capacity, the algorithm checks every item.

If an item fits into the current capacity:

$dp[w] = \max(dp[w], dp[w - \text{weight}[i]] + \text{value}[i])$

The use of $dp[w - \text{weight}[i]]$ allows the same item to be selected again. This is what makes the algorithm suitable for the Unbounded Knapsack problem.

Algorithm

1. Read the number of items.
2. Read the values of the items.
3. Read the weights of the items.
4. Read the knapsack capacity.
5. Create a DP array of size $W + 1$.
6. Initialize all values to zero.
7. For every capacity from 0 to W :
 - Check every item.
 - If the item fits, calculate the new profit.
 - Store the maximum profit.
8. Print $dp[W]$.

Pseudocode

UNBOUNDED-KNAPSACK(n , values, weights, W)

Create $dp[0 \dots W]$

Initialize all dp values to 0

FOR $w = 0$ TO W

FOR $i = 0$ TO $n-1$

IF $weights[i] \leq w$

$dp[w] = \text{MAX}(\$

$dp[w],$

$dp[w - weights[i]] + values[i]$

$)$

RETURN $dp[W]$

Example Calculation

Given:

Values = {10, 30, 20}

Weights = {5, 10, 15}

Capacity = 100

The second item has:

Weight = 10

Profit = 30

Since it can be selected repeatedly:

Number of items = $100 / 10$

= 10

Therefore:

Total Profit = 10×30

= 300

Hence:

Maximum Profit = 300

DP Recurrence

The recurrence relation is:

$dp[w] = \max(dp[w], dp[w - \text{weight}[i]] + \text{value}[i])$

where:

- w = Current capacity.
- $\text{weight}[i]$ = Weight of item i .

- $\text{value}[i] = \text{Profit of item } i.$
- $\text{dp}[w] = \text{Maximum profit for capacity } w.$

Time Complexity

There are:

- $W + 1$ possible capacities.
- n different item types.

For each capacity, all items are checked.

Therefore:

Time Complexity = $O(n \times W)$

For the sample:

$$n = 3$$

$$W = 100$$

$$\text{Operations} \approx 3 \times 100 = 300$$

Space Complexity

The algorithm uses a one-dimensional DP array:

$\text{dp}[0 \dots W]$

The size of this array is $W + 1$.

Therefore:

Space Complexity = $O(W)$

This is more space-efficient than using a two-dimensional DP table.

Result

The Unbounded Knapsack problem was successfully solved using Dynamic Programming.

For:

$n = 3$

Values = 10 30 20

Weights = 5 10 15

$W = 100$

the program produces:

Maximum Profit = 300